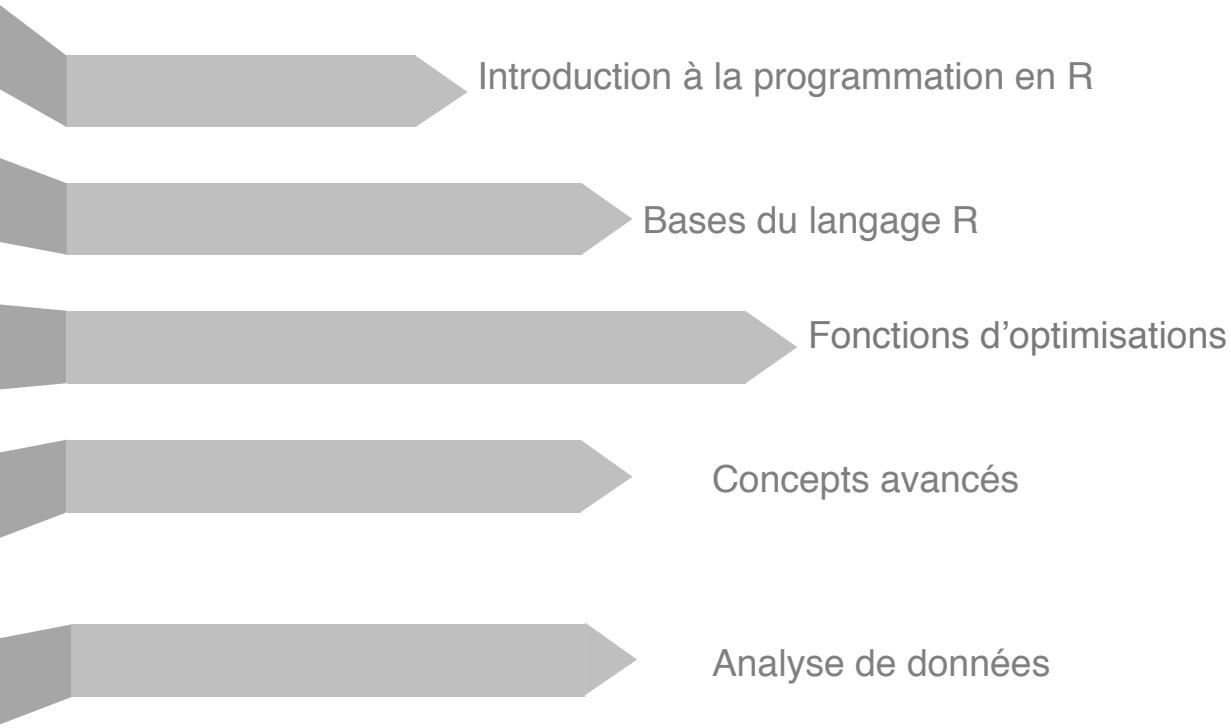
Université Abdel Malek Essaâdi
Faculté des Sciences et Techniques
Département Génie Informatique
Tanger

Big Data Analytics With R

Pr. EZZIYYANI Mostafa

BOTEIN POWERPOINT

Plan du cours



Introduction



Introduction à la programmation en R



Le R est un langage particulièrement puissant pour les applications mathématiques et statistiques puisque précisément développé dans ce but.

Parmi ses caractéristiques particulièrement intéressantes, on note :

- langage basé sur la notion de vecteur, ce qui simplifie les calculs mathématiques.
- pas de typage ni de déclaration obligatoire des variables ;
- programmes courts, en général quelques lignes de code seulement ;
- temps de développement très court.

Introduction à la programmation en R

Certains langages d'usage général, comme le C, Java ou VBA, visent une grande variété d'applications.

Le langage R, en revanche, a été développé spécifiquement pour les calculs mathématiques et statistiques, ainsi que pour la manipulation de données.

Il ne faut donc pas se surprendre si le langage respecte les principales règles des mathématiques et en particulier l'algèbre vectorielle et matricielle.



bases du langage R

Les variables en R

Les variables dans R sont non typées

Lorsque l'on dit qu'une variable n'est pas typée en R, cela signifie qu'une même variable peut contenir des données de types différents, à l'intérieur d'une même fonction ou si vous préférez d'un même programme.

Déclaration des variables

Il n’est pas non plus nécessaire de déclarer ou d’initialiser les variables avant de les utiliser. Le programmeur habitué à travailler en Java ou en VBA pourrait avoir tendance à initialiser ses variables au début de ses fonctions R. C’est une pratique à éviter. Il n’est pas nécessaire de déclarer ses variables en R et le faire est considéré comme du mauvais style.

Les premières opérations

Affectation

- `a <- 1.2` **a** est une variable, en interne c'est un vecteur de taille 1, et il peut contenir des données numériques, le typage est donc automatique. Dans la terminologie R, **a** est un « objet ».

```
> A<-5
> A
[1] 5
> B<- A
>B
[1] 5
```

Autres possibilités

- `A<- 1;b<-5 ;d<- a + b;`

Le saisie et l'affichage

Saisie

```
A<- scan()
B<- scan()
D<- a + b
```

(1) R est interprété, on peut modifier le code source et relancer.

(2) Utiliser la fonction scan() qui permet d'effectuer une saisie console lors de l'exécution du programme. C'est l'approche que nous privilégierons dans un premier temps

Affichage

```
D<-  a + b
d                #Affichage implicite
print(d)         #Affichage explicite
```

N.B. L'affichage implicite ne marche pas dans les procédures et fonctions L'affichage explicite marche toujours !

Les types de données

- Logical

```
> x <- T; y <- F
> x; y
[1] TRUE
[1] FALSE
```

- Numerical

```
> a <- 5; b <-
sqrt(2)
> a; b
[1] 5
[1] 1.414214
```

- Character

```
> a <- "1"; b <- 1
> a; b
[1] "1"
[1] 1
> a <- "character"
> b <- "a"; c <- a
> a; b; c
[1] "character"
[1] "a"
[1] "character"
```

Les opérations mathématiques

```
> 5 + (6 + 7) * pi^2
[1] 133.3049
> log(exp(1))
[1] 1
> log(1000, 10)
[1] 3
> sin(pi/3)^2 + cos(pi/3)^2
[1] 1
> Sin(pi/3)^2 + cos(pi/3)^2
Error: couldn't find function "Sin"
```

Les vecteurs

- En R, à toutes fins pratiques, *tout* est un vecteur. Contrairement à certains autres langages de programmation, il n'y a pas de notion de scalaire en R ; un scalaire est simplement un vecteur de longueur 1. le vecteur est l'unité de base dans les calculs.
- Dans un vecteur simple, tous les éléments doivent être du même mode. Nous nous restreignons à ce type de vecteurs pour le moment.

Les vecteurs

une collecte ordonnée de données du même type de données

```
> x <- c(5.2, 1.7, 6.3)
> log(x)
[1] 1.6486586 0.5306283 1.8405496
> y <- 1:5
> z <- seq(1, 1.4, by = 0.1)
> y + z
[1] 2.0 3.1 4.2 5.3 6.4
> length(y)
[1] 5
> mean(y + z)
[1] 4.2
```

Les matrices

- Une matrice est un vecteur avec un attribut dim de longueur 2. Cela change implicitement la classe de l'objet pour « matrix » et, de ce fait, le mode d'affichage de l'objet ainsi que son interaction avec plusieurs opérateurs et fonctions.

La fonction de base pour créer des matrices est matrix

```
> matrix(1:6, nrow = 2, ncol = 3)
```

table rectangulaire de données du même type

```
> m <- matrix(1:12, 4, byrow = T); m
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
[4,]   10   11   12
> y <- -1:2
> m.new <- m + y
> t(m.new)
     [,1] [,2] [,3] [,4]
[1,]    0    4    8   12
[2,]    1    5    9   13
[3,]    2    6   10   14
> dim(m)
[1] 4 3
> dim(t(m.new))
[1] 3 4
```

les listes

La liste est le mode de stockage le plus général et polyvalent du langage R. Il s'agit d'un type de vecteur spécial dont les éléments peuvent être de n'importe quel mode, y compris le mode list. Cela permet donc d'emboîter des listes, d'où le qualificatif de récursif pour ce type d'objet.

La fonction de base pour créer des listes est list:

```
(x <- list(size = c(1, 5, 2), user = "Joe", new = TRUE))
 $size [1] 1 5 2
 $user [1] "Joe"
 $new [1] TRUE
```

La création d’objets de classe ”list”

exemple

```
R> l2 <- list(minuscules = letters, majuscules = LETTERS, mois = month.name)
L2
```

```
minuscules
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
[20] "t" "u" "v" "w" "x" "y" "z"

$majuscules
 [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
[20] "T" "U" "V" "W" "X" "Y" "Z"

$mois
 [1] "January"   "February"  "March"     "April"     "May"       "June"
 [7] "July"      "August"    "September" "October"   "November"  "December“
```

```
-R> length(l2)
[1]  3
-R>names(l2)
[1] "minuscules" "majuscules" "mois"
```

•Il existe trois grands éléments de contrôle permettant de manipuler les objets et les fonctions de manière à programmer une ou des tâches :

- for
- if
- while

•Tous les trois s'utilisent à peu prés de la même manière et peuvent se combiner les uns aux autres de manière imbriquée :

```
 for/if/while          (conditions)
{
liste d'instructions
}
```

La boucle for

Très utile quand vous devez répéter une ou plusieurs tâches un nombre important de fois :

```
> res <- c()
> for (i in 1:6)
    {
    print(i)
    res <-c(res,i^2)
    }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
```

```
>res
[1] 1 4 9 16 25 36
```

La possibilité de jouer avec les itérations

On peut aussi créer son propre index de valeurs prisent par *i* :

```
> index   <-rev(res)
> index
[1]  36  25  16  9  4  1
> res  <-c()
> for  (i  in  index)
{
print(i)
res      <-c(res,sqrt(i))
}
  [1] 36
  [1] 25
  [1] 16
  [1]  9
  [1]  4
  [1]  1
>res
          [1] 6  5  4  3  2  1
```

La condition if, la décision

Très utile pour exécuter ou non une ou plusieurs tâches si une condition est observée :

```
>  seuil  <-  0
>  x  <-  2
>  if  (x>seuil)
 }
 print(x^2+2*x+1)
 }
[1]   9
>  x  <-  -x
>  if  (x>seuil)
 {
 print(x^2+2*x+1)
 }
>
```

Les conditions

Dans le cas de plusieurs conditions, on peut emboîter le processus décisionnel à l'aide de if/else :

```
>  seuil1  <- 0
>  seuil2  <- 5
>  x <- -1
>  if(x>seuil2)
 {
 print(x^2)
 }      else  if(x>seuil1)
     {
     print(x^2+2*x+1)
     }      else  print("impossible      : valeur  négative  de  x")
[1]  "impossible     : valeur  négative  de  x"
```

Trés utile quand vous devez répéter une ou plusieurs tâches tant que la condition est vraie :

```
>  Res <-c()
>  i  <-1
>  While (i<7)
 {
 Res <- c(res, i^2)
 i <- i+1
 }
>  i
     [1]  7
>  res
[1]   1  4  9  16  25  36
```

La data frame

- Un est une liste de classe « **data.frame** » dont tous les éléments sont de la même longueur (ou comptent le même nombre de lignes si les éléments sont des matrices).

- Il est généralement représenté sous la forme d'un tableau à deux dimensions. Chaque élément de la liste sous-jacente correspond à une colonne.

- Bien que visuellement similaire à une matrice un *data frame* est plus général puisque les colonnes peuvent être de modes différents ; pensons à un tableau avec des noms (mode **character**) dans une colonne et des notes (mode **numeric**) dans une autre.

La data frame(le type data frame)

- liste de vecteurs de données
- tous les vecteurs sont de même longueur
- certains sont numériques, d'autres des factor, des étiquettes, etc.
- les vecteurs peuvent être nommées = nom des variables
- les lignes peuvent être nommés = étiquettes des observations
- on peut la manipuler comme une matrice, avec 2 indices **[**n°ligne**,** n°colonne**]**

La data frame

```
#création à partir de 3 vecteurs de valeurs

V1 <-c(15,8.2,14)
v2 <-c(TRUE,FALSE,TRUE)
v3 <-factor(c("M","F","M"))
#liste
liste <-list(v1,v2,v3)
#data.frame
donnees<-data.frame(liste)
#nommer les colonnes et les lignes
colnames(donnees) <-c("x1","x2","x3")
rownames(donnees) <-c("pierre","paul","jacques")
#affichage
print(class(donnees))
print(summary(donnees))
print(donnees)
```

```
> print(class(donnees))
[1] "data.frame"
> print(summary(donnees))
       x1           x2          x3
 Min.   : 8.2   Mode :logical   F:1
 1st Qu.:11.1   FALSE:1         M:2
 Median :14.0   TRUE :2
 Mean   :12.4   NA's :0
 3rd Qu.:14.5
 Max.   :15.0
> print(donnees)
          x1    x2 x3
pierre  15.0  TRUE  M
paul     8.2 FALSE  F
jacques 14.0  TRUE  M
```

La data frame(accès au variables)

```
#accès 1ere variable
#par nom
donnees$x1
#accès indicé
#par num.colonne
donnees[1]
#format liste
donnees[[1]]
#format matrice
donnees[,1]
#accès par nom
donnees["x1"]
donnees[["x1"]]
donnees[,"x1"]
```

```
> #accès à la première variable
> #par nom
> donnees$x1
[1] 15.0  8.2 14.0
> #accès indicé
> #par num.colonne
> donnees[1]
          x1
pierre  15.0
paul     8.2
jacques 14.0
> #format liste
> donnees[[1]]
[1] 15.0  8.2 14.0
> #format matrice
> donnees[,1]
[1] 15.0  8.2 14.0
> #accès par nom
> donnees["x1"]
          x1
pierre  15.0
paul     8.2
jacques 14.0
> donnees[["x1"]]
[1] 15.0  8.2 14.0
> donnees[,"x1"]
[1] 15.0  8.2 14.0
```

Example:

data frame: is supposed to represent the typical data table that researchers come up with – like a spreadsheet.

It is a rectangular table with rows and columns; data within each column has the same type (e.g. number, text, logical), but different columns may have different types.

> maDFListe ??

	localisation	tumorsize	progress
XX348	proximal	6.3	FALSE
XX234	distal	8.0	TRUE
XX987	proximal	10.0	FALSE

```
> v1<-c('proximal','distal','proximal')
> v2<-c(6.3,8,10)
> V3<-c(FALSE,TURE,FALSE)
> maListe<-list(v1,v2,v3)
> maDFListe<-data.frame(maListe)
> colnames(maDFListe)<-
c('Loclisation','Tumeursize','progress')
> rownames(maDFListe)<-c('A','B','C')
> attach(maDFliste)
> maDFListe
```

Solution :

```
> v1<-c('proximal','distal','proximal')
> v2<-c(6.3,8,10)
> V3<-c(FALSE,TURE,FALSE)
> maListe<-list(v1,v2,v3)
> maDFListe<-data.frame(maListe)
> colnames(maDFListe)<-c('Loclisation','Tumeursize','progress')
> rownames(maDFListe)<-c('A','B','C')
> attach(maDFliste)
> maDFListe
> a<-maDFListe
```

Subsetting

Individual elements of a vector, matrix, array or data frame are accessed with "[ ]" by specifying their index, or their name

```
> a
       localisation tumorsize progress
XX348    proximal       6.3        0
XX234      distal       8.0        1
XX987    proximal      10.0        0
> a[3, 2]
[1] 10
> a["XX987", "tumorsize"]
[1] 10
> a["XX987",]
       localisation tumorsize progress
XX987    proximal        10        0
```

Subsetting

```
> a
       localisation tumorsize progress
XX348    proximal       6.3        0
XX234      distal       8.0        1
XX987    proximal      10.0        0
> a[c(1,3),]
       localisation tumorsize progress
XX348    proximal       6.3        0
XX987    proximal      10.0        0
> a[c(T,F,T),]
       localisation tumorsize progress
XX348    proximal       6.3        0
XX987    proximal      10.0        0
> a$localisation
[1] "proximal" "distal"   "proximal"
>  a$localisation=="proximal"
[1]  TRUE FALSE  TRUE
> a[ a$localisation=="proximal", ]
       localisation tumorsize progress
XX348    proximal       6.3        0
XX987    proximal      10.0        0
```

subset rows by a vector of indices

subset rows by a logical vector

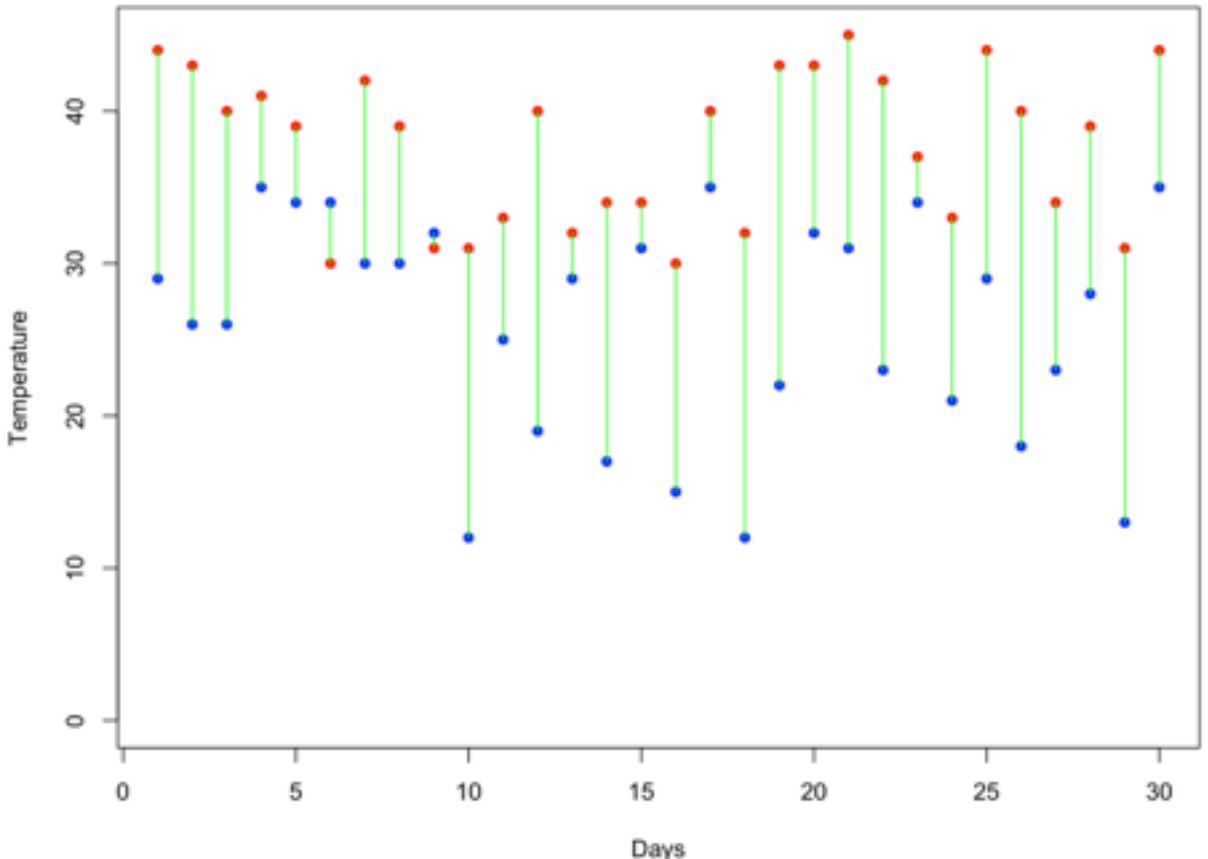
subset a column

comparison resulting in logical vector

subset the selected rows

La data frame : Exemple

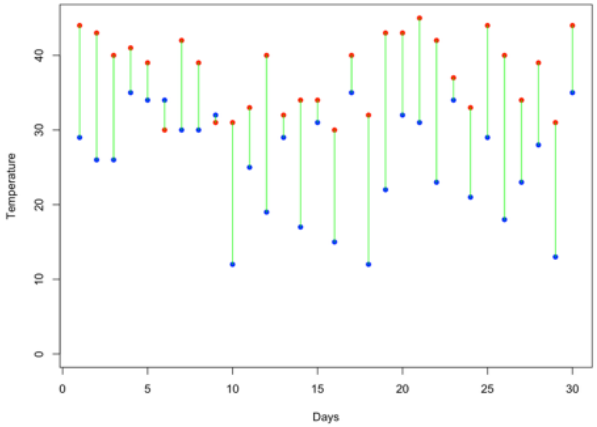
Représentation de la temperature max et min d'un mois



La data frame : Exemple

Représentation de la temperature max et min d'un mois

```
Days<-seq(1,30,1)
Tmin<-sample(12:35,rep=TRUE,30)
Tmax<-sample(30:45,rep=TRUE,30)
jantemps <- list(Days,Tmin,Tmax)
DFjantemps <-data.frame(jantemps)
DFJanTemps<-data.frame(jantemps)
colnames(DFJanTemps)<-c('D','Tmi','Tma')
rownames(DFJanTemps)<-seq(1:30)
points(DFJanTemps$D,DFJanTemps$Tmi,col="blue",pch=10)
points(DFJanTemps$D,DFJanTemps$Tma,col=« red",pch=10)
for( i in 1:30 ) lines( c(i,i), c(DFJanTemps$Tmi[i],DFJanTemps$Tma[i]),col="green")
```



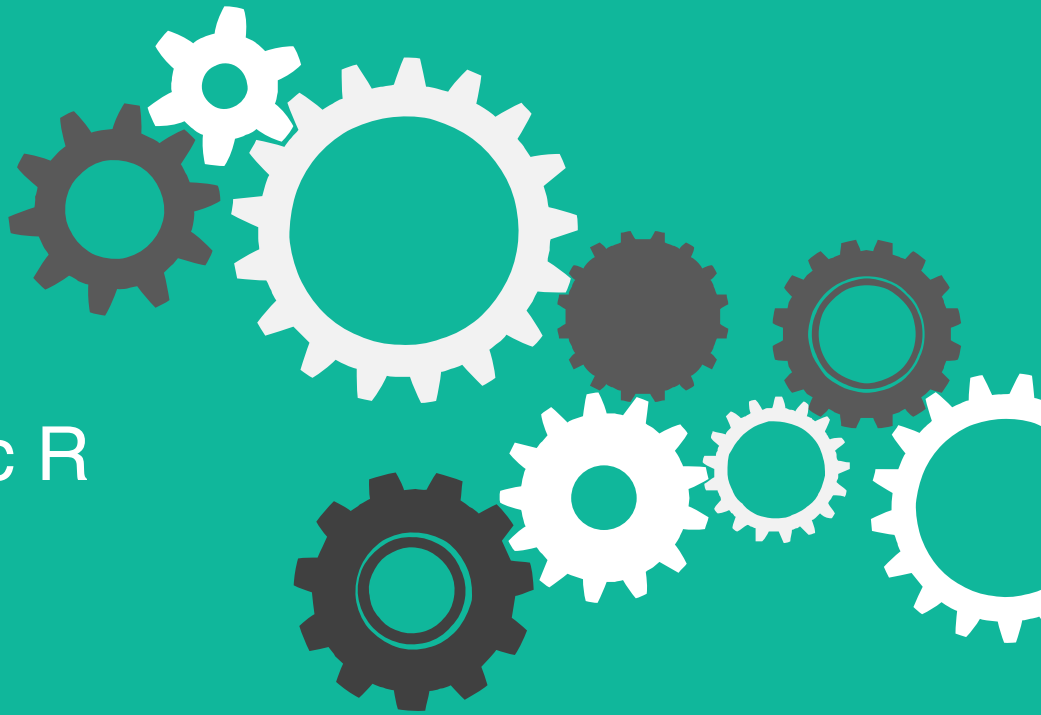
Les commandes sous R

Opération	Commande sous R
pi	pi
Racine carrée	sqrt
Multiplication, division, addition, soustraction	*, /, +, -
Arrondir	round
Moyenne, médiane, variance	mean, median, var
Obtenir pour chaque variable d'un tableau : quartiles et moyenne	summary
Ecart-type	sd
Etendue d'une série de valeurs	range
Appliquer une même fonction (FUN) à toutes les catégories Y d'une variable X	tapply (X,Y,FUN)
Importer des données d'un tableau	read.delim, read.table
Créer une matrice	matrix
Effectuer un modèle linéaire	lm(x~y*z)
Représenter un nuage de points	plot(y~x)
Tracer une droite de régression correspondant à un modèle linéaire (lm)	abline(lm)
Fabriquer un graphique en « boîte à moustache »	boxplot(x~y)
Options graphiques (propriétés de la fenêtre, des polices des axes,...)	par

BOTEIN POWERPOINT

Fonctions avec R





Les Fonction avec R

1. fonction <- function(a,b=1) { listecommandes
   return(nom1=resultat1,nom2=resultat2) }

Fonction avec 2 arguments en entrée a et b.

b a comme valeur par défaut 1 renvoyant une liste a` deux composantes (nommées nom1 et nom2).

2. fonction(3,1) ou fonction(3)

3. res <- fonction(3,2) appel de la fonction

Les Fonction avec R

```
Additionne <- function( a, b )
{
# On additionne les valeurs a et b
c <- a + b
# Imprime le résultat de l'addition
cat( c , "\n")
}
```

Les Fonction avec R

Comment retourner les résultats d’une fonction
Toutes les commandes que vous utilisez et inscrivez dans la console R peuvent être intégrées dans le script d’une fonction. Voyons ici une fonction un peu plus complexe qui effectue des tâches mathématiques et renvoie une valeur :

Les Fonction avec R

```
NbPneu <- function(voiture=0, bicyclette=0, monocycle=0, tricycle=0)
{
    # Dans cette fonction, nous calculons le nombre total de pneus
    A <- voiture * 4
    # Le Nb de pneus de voiture
    B <- tricycle * 3
    # Le Nb de pneus de tricycle
    C <- bicyclette * 2
    # Le Nb de pneus de bicyclette
    D <- monocycle * 1
    # Le Nb de pneus de monocycle
    NbPneuTotal <- A + B + C + D
    # Retourne le nombre total de pneus
    return( NbPneuTotal )
}
```

NbPneu(voiture=2, bicyclette=0, monocycle=0, tricycle=1)

Les Fonction avec R

```
CoefVar <- function( datum )
{
    # Fonction permettant de calculer le coefficient de variation
    # Applicable seulement pour des données ayant un véritable zéro
    # Calcul de l'écart type
    EcartType <- sd(datum)
    # Calcul de la moyenne
    Moyenne <- mean(datum)
    # Calcul du coefficient de variation
    CV <- 100 * EcartType / Moyenne
    # Retourne le résultat
    return(CV)
}
```

```
TestData <- c( 1, 2, 3, 4, 5 )
CoefVar( datum = TestData )
```

Les Fonction avec R

```
CentrageReduction <- function( x )
{
    # Cette fonction centre et réduit les données d'un vecteur
    # nommé x. Elle effectue la même tâche que la fonction «scale».
    # Soustrait la moyenne au vecteur x
    xCentre<- x - mean( x )
    # Calcule la variance
    variance <- var( x )
    # Trouve le l'écart type
    EcartType<- variance ^ (1/2)
    # Complète le centrage et la réduction des données
    xCentreReduit <- xCentre / EcartType
    # Retourne le vecteur de données centrées et réduites
    return( xCentreReduit )
}
```

```
vec<- c( 4, 12, 31, 4,25,8,33,6,9 )
CentrageReduction( x = vec )
```

fonctions d’optimisations



méthodes d'optimisation

Les **méthodes d'optimisation** sont des méthodes numériques pour chercher le minimum ou le maximum d'une fonction.

Dans le cas d'une optimisation sans contrainte, R propose la commande `optimise()` ou `optimize()` (synonymes). Sa syntaxe globale est :

```
optimize(fct, intervalle)
optimize(fct, intervalle, maximum = TRUE)
optimize(f = fct, interval = intervalle)
```

où

`fct` est la fonction à minimiser,

`intervalle` est l'intervalle sur lequel il effectue la recherche ;

`maximum` est un booléen : « vrai » si l'on cherche la maximum, « faux » (valeur par défaut) si l'on cherche le minimum.

exemple

Soit f une fonction tel que f(x)=sin(x)/x

On défini la fonction dans le consol comme suite:

```
> f <- function(x) sin(x)/x
```

Pour chercher le minimum de f sur l'intervalle [-π ; π]:

```
> optimize(f, c(-pi, pi))
```

Le résultat vaut:

```
$minimum [1] 3.141535
$objective [1] 1.824484e-05
```

Ou `$minimum` est la valeur de x et `$objective` représente f(x)

Pour stoker le resultat de la fonction `optimize()` dans une variable:

```
> Minf <- optimize(f, c(-pi, pi))
> print(minf)


$minimum [1] -0.999948
$objective [1] -16177.2
```

Exemple2(Fonction avec paramètre)

Si la fonction possède des paramètres, on doit les passer en utilisant leur nom exact.
Par exemple, si la fonction est

$$f_{a,\,b,\,c}(x, y) = (x - a)^2 + (y - b)^2 + c$$

alors le script suivant permet de trouver le minimum :

```
f <- function(x, a, b, c) (x[1] - a)^2 + (x[2] - b)^2 + c x0 <- c(0, 0)
solution <- optim(x0, f, a = 5, b = 4, c = -25) print(solution$par)
print(solution$value)
```

Ce qui donne

```
[1] 4.999483 4.000116
[1] -25
```

La méthode d'optimisation avec contrainte linéaire

On recherche le minimum de *f*, mais avec des contraintes linéaires, de type :
inégalités : ui×xi ≥ ci
Le logiciel R propose la commande constrOptim(), qui fait appel au
solveur optim() mais en ajoutant une fonction de pénalité pour gérer les contraintes.
La syntaxe de base est :
constrOptim(x0, f, NULL, u, c) constrOptim(par = x0, fn = f, ui = u, ci = c) où

- x0 est le vecteur de départ, qui doit être dans la zone de faisabilité, c'est-à-dire vérifier les inégalités de contrainte ;
- f est la fonction ;
- NULL indique que l'on ne définit pas la fonction gradient ;
- u et c sont des vecteurs représentant les contraintes.

Par exemple, si l'on veut rechercher le minimum de la fonction y = sin(x) sur l'intervalle [-π ; π] :
la fonction f est sin
les contraintes linéaires sont
1×x ≥ -π,
-1×x ≥ -π
donc on a en notation matricielle u×x =c

Exemple2(Fonction avec paramètre)

si l'on veut rechercher le minimum de la fonction $f(x) = \sin(x)$ sur l'intervalle $[-\pi ; \pi]$:

- la fonction f est sin
- les contraintes linéaires sont
  $1 \times x \geq -\pi$,
  $-1 \times x \geq -\pi$
  donc on a en notation matricielle $\mathbf{u} \times x = \mathbf{c}$ avec
- on part de $x_0 = 0$.

```
x0 <- matrix(0, 1, 1)
f <- sin
u <- matrix(c(1, -1), 2, 1)
c <- matrix(c(-pi, -pi), 2, 1)
solution <- constrOptim(x0, f, NULL, u, c)
```

Concepts avancés





Objectifs

- Passer des valeurs à une fonction via l'argument '...'.

- Effectuer des sommaires sur des tableaux à l'aide de la fonction "apply".

- Réduire des listes avec les fonctions "lapply", "sapply" et "mapply" ; comparer l'effet de ces fonctions.

Argument ”...”

- Cet argument signifie qu’une fonction peut accepter un ou plusieurs arguments autres que ceux faisant partie de sa définition.

Le contenu de l’argument ‘...’ n’est ni pris en compte, ni modifié par la fonction. Il est généralement simplement passé tel quel à une autre fonction qui, elle, saura traiter les arguments qui lui sont ainsi passés

- Pour plus de détails on va maintenant voir les définitions des fonctions “apply”, “lapply”, “sapply” et “mapply”.

Fonction Apply

La fonction apply sert à appliquer une fonction quelconque sur une partie d'une matrice ou, plus généralement, d'un tableau. La syntaxe de la fonction est la suivante :

apply(X, MARGIN, FUN, ...)

où
X est une matrice ou un tableau ;
MARGIN est un vecteur d'entiers contenant la ou les dimensions de la matrice ou du tableau sur lesquelles la fonction doit s'appliquer ;
FUN est la fonction à appliquer ;
'...' est un ensemble d'arguments supplémentaires, séparés par des virgules, à passer à la fonction FUN.

Exemples de fonction Apply

(x <- matrix(sample(1:100, rep = TRUE), 5, 4))

Pour la declaration d'une matrice de cinq lignes et quatre colonnes avec répétition

```
> (x <- matrix(sample(1:100, rep = TRUE), 5, 4))
     [,1] [,2] [,3] [,4]
[1,]   35   99   63   59
[2,]   95   33   15   22
[3,]   88   64   47   99
[4,]   55   99   29   61
[5,]   68    8   80   93
>
```

Exemples de fonction Apply

- Cet argument signifie qu'une fonction peut accepter un ou plusieurs arguments autres que ceux faisant partie de sa définition.

Le contenu de l'argument '...' n'est ni pris en compte, ni modifié par la fonction. Il est généralement simplement passé tel quel à une autre fonction qui, elle, saura traiter les arguments qui lui sont ainsi passés

- Pour plus de détails on va maintenant voir les définitions des fonctions "apply", "lapply", "sapply" et "mapply".

Exemples de fonction Apply

apply(x, 2, min) le minimum par colomne

```
> apply(x, 2, min)
[1] 35  8 15 22
>
```

Apply(x,1,mean) la moyenne par ligne

```
> apply(x, 1, mean)
[1] 64.00 41.25 74.50 61.00 62.25
>
```

Exemples de fonction Apply

Puisqu'il n'existe pas de fonctions internes pour effectuer des sommaires sur des tableaux, il faut toujours utiliser la fonction apply.
Si X est un tableau de plus de deux dimensions, alors l'argument passé à FUN peut être une matrice ou un tableau.
Lorsque X est un tableau à trois dimensions et que MARGIN est de longueur 1, cela équivaut à appliquer la fonction FUN sur des « tranches » (des matrices) de X.

Exemples de fonction Apply

Déclaration de quatre tableaux de trois lignes trois colonnes avec répétition

(x <- array(sample(1:10,rep = TRUE), c(3, 3, 4)))

```
> (x <- array(sample(1:10, rep = TRUE), c(3, 3, 4)))
, , 1

     [,1] [,2] [,3]
[1,]    6    7   10
[2,]   10    8    3
[3,]    8    9    5

, , 2

     [,1] [,2] [,3]
[1,]    2    8    9
[2,]    6    7   10
[3,]   10    8    3

, , 3

     [,1] [,2] [,3]
[1,]    5   10    8
[2,]    2    8    9
[3,]    6    7   10

, , 4

     [,1] [,2] [,3]
[1,]    3    6    7
[2,]    5   10    8
[3,]    2    8    9

>
```

Exemples de fonction Apply

apply(x, 1, sum)
#Somme tranches horizontales

```
> apply(x, 1, sum)
[1] 81 86 85
> 
```

apply(x, c(1, 2), sum)
#Somme sur les colonnes et lignes de la matrice

```
> apply(x, c(1, 2), sum)
     [,1] [,2] [,3]
[1,]   16   31   34
[2,]   23   33   30
[3,]   26   32   27
> 
```

Fonctions Lapply et Sapply

Les fonctions lapply et sapply sont similaires à la fonction apply en ce qu'elles permettent d'appliquer une fonction aux éléments d'une structure

— le vecteur ou la liste en l'occurrence. Leur syntaxe est similaire :

lapply(X, FUN, ...) sapply(X, FUN, ...)

La fonction lapply applique une fonction FUN à tous les éléments d'un vecteur ou d'une liste X et retourne le résultat sous forme de liste. Le résultat est donc :

list(FUN(X[[1]], ...),
FUN(X[[2]], ...),
FUN(X[[3]], ...),

Exemples de fonction Lapply

Une liste formée de quatre vecteurs aléatoires de taille 5, 6, 7 et 8 :

(x <- lapply(5:8, sample, x = 1:10))

```
> (x <- lapply(5:8, sample, x = 1:10))
[[1]]
[1]  7 10  6  1  5

[[2]]
[1] 2 9 3 6 1 8

[[3]]
[1]  9  4  6  7 10  5  2

[[4]]
[1] 10  6  5  2  8  4  7  3

> |
```

Exemples de fonction Apply

lapply(x, mean)

#calculer la moyenne de chacun des vecteurs

```
> lapply(x, mean)
[[1]]
[1] 5.8

[[2]]
[1] 4.833333

[[3]]
[1] 6.142857

[[4]]
[1] 5.625
```

Exemples de fonction Sapply

La fonction Sapply est similaire à Lapply, sauf que le résultat est retourné sous forme de vecteur

sapply(x, mean)

```
> sapply(x, mean)
[1] 5.800000 4.833333 6.142857 5.625000
>
```

Exemples de fonction Lapply Sapply

```
(x <- lapply(rep(5, 3), sample, x = 1:10))
#Déclaration de trois listes de cinques vecteurs
```

```
> (x <- lapply(rep(5, 3), sample, x = 1:10))
[[1]]
[1] 7 8 5 1 2

[[2]]
[1] 9 6 1 2 8

[[3]]
[1]  2  7  4 10  8

>
```

```
sapply(x, sort)
# Pour faire le tri
```

```
> sapply(x, sort)
     [,1] [,2] [,3]
[1,]    1    1    2
[2,]    2    2    4
[3,]    5    6    7
[4,]    7    8    8
[5,]    8    9   10
>
```

La fonction Mapply

- La fonction mapply est une version multidimensionnelle de sapply. Sa syntaxe est, essentiellement,

  mapply(FUN, ...)

- Le résultat de mapply est l'application de la fonction FUN aux premiers éléments de tous les arguments contenus dans '...', puis à tous les seconds éléments, et ainsi de suite.

- Ainsi, si v et w sont des vecteurs, mapply(FUN, v, w) retourne sous forme de liste, de vecteur ou de matrice, selon le cas, FUN(v[1], w[1]), FUN(v[2], w[2]), etc.

Exemples de fonction Mapply

mapply(rep, 1:4, 4:1)

#Les éléments de '...' sont recyclés au besoin.

```
> mapply(rep, 1:4, 4:1)
[[1]]
[1] 1 1 1 1

[[2]]
[1] 2 2 2

[[3]]
[1] 3 3

[[4]]
[1] 4

>
```

Exemples de fonction Mapply

mapply(seq, 1:6, 6:8)

```
> mapply(seq, 1:6, 6:8)
[[1]]
[1] 1 2 3 4 5 6

[[2]]
[1] 2 3 4 5 6 7

[[3]]
[1] 3 4 5 6 7 8

[[4]]
[1] 4 5 6

[[5]]
[1] 5 6 7

[[6]]
[1] 6 7 8

>
```

Analyses de données



Importation et exportation de données

- Read data from and write data to
- I R native formats (incl. Rdata and RDS)
- I CSV files
- I EXCEL files
- I ODBC databases
- I SAS databases

Sauvegarde et surcharge des objets

save(): save R objects into a .Rdata file

load(): read R objects from a .Rdata file

rm(): remove objects from R

```
a<-1:10
save(a,file="./data/dumData.Rdata")
rm(a)
a

## Error: object 'a' not found

load("./data/dumData.Rdata")
a

##[1] 1 2 3 4 5 6 7 8 9 10
```

Import from and Export to .CSV Files

- **write.csv():** write an R object to a .CSV file
- **read.csv():** read an R object from a .CSV file

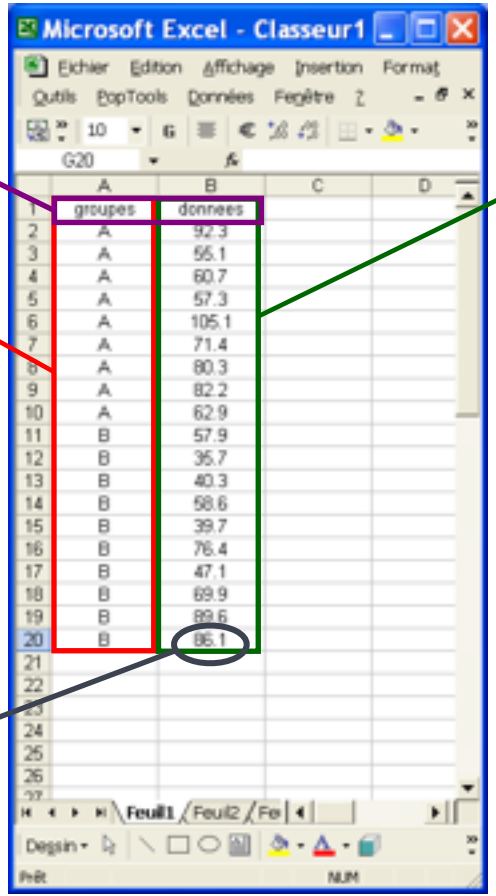
```
# create a data frame
var1<-1:5
var2<-(1:5)/10
var3<-c("R","and","Data Mining","Examples","Case Studies")
df1<-data.frame(var1, var2, var3)
names(df1)<-c("VarInt","VarReal","VarChar")
# save to a csv file
write.csv(df1,"./data/dummmyData.csv",row.names=FALSE)
# read from a csv file
df2<-read.csv("./data/dummmyData.csv")
print(df2)
```

```
##   VarInt  VarReal       VarChar
## 1      1      0.1             R
## 2      2      0.2           and
## 3      3      0.3   Data Mining
## 4      4      0.4      Examples
## 5      5      0.5  Case Studies
```

- Chaque variable doit être présentée en colonne

Noms de colonnes

Variable explicative



Variable mesurée

- Pas de colonne vide à gauche
- Pas de ligne vide au-dessus
- Uniquement le tableau de données!

1. Pas d'espace
2. Pas d'accent
3. Pas de caractères spéciaux

Privilégier les points comme séparateur de décimale

Enregistrer le fichier de données en format

texte (séparateur:tabulation) (*.txt) **pour Microsoft Office**

texte (*.csv) **pour Open Office**

Package xlsx : read, write, format Excel 2007 and Excel 97/2000/XP/2003 files

write.xlsx(): write an R object to a .Excel

read. xlsx(): read an R object from a . Excel

```
library(xlsx)
xlsx.file<-"./data/dummmyData.xlsx"
write.xlsx(df2, xlsx.file,sheetName="sheet1",row.names= F)
df3<-read.xlsx(xlsx.file,sheetName="sheet1")
df3
```

```
##      VarInt  VarReal         VarChar
## 1         1      0.1               R
## 2         2      0.2             and
## 3         3      0.3   Data Mining
## 4         4      0.4        Examples
## 5         5      0.5  Case Studies
```

Read from Databases

Package RODBC : provides connection to ODBC databases.

-Function odbcConnect(): sets up a connection to database

- **sqlQuery()**: sends an SQL query to the database

- **odbcClose()** :closes the connection

```
library(RODBC)
db<-odbcConnect(dsn="servername",uid="userid",
                pwd="******")
sql<-"SELECT * FROM lib.table WHERE ..."
# or read query from file
sql<-readChar("myQuery.sql",nchars=99999)
myData<-sqlQuery(db, sql,errors=TRUE)
odbcClose(db)
```

Functions **sqlFetch()**, **sqlSave()** and **sqlUpdate()**: read, write or update a table in an ODBC database

On peut avoir besoin d'exporter un tableau de données dans **R** vers un fichier dans différents formats.

La plupart des fonctions d'import disposent d'un équivalent permettant l'export de données. On citera notamment :

• write_csv , write_delim , write_tsv (**readr**)permettent d'enregistrer un *data frame* ou un tibble dans un fichier au format texte délimité

• write_sas (**haven**) permet d'exporter au format **SAS**

• write_sav (**haven**) permet d'exporter au format **SPSS**

• write_dta (**haven**) permet d'exporter au format **Stata**

Si vous travaillez sur des données de grandes dimensions, les formats texte peuvent être lents à exporter et importer. Dans ce cas, l'extension **feather** peut être utile : elle permet d'enregistrer un *data frame* au format feather, qui n'est pas le plus compact mais qui est extrêmement rapide à lire et écrire 1, page 01.

Les fonctions read_feather et write_feather permettent d'importer et exporter des tableaux de données dans ce format.

Type de fichier souhaité	Fonction	Extension
texte	`write.table`	utils
CSV	`write.csv`	utils
CSV	`write_csv`	readr
Excel	`write.xlsx`	xlsx
dBase	`write.dbf`	foreign
SPSS	`write_sav`	haven
SPSS	`write.foreign`	foreign
Stata	`write.dta`	foreign
Stata	`write_dta`	haven
SAS	`write.foreign`	foreign
SPSS	`write.foreign`	foreign